

Principal Component Analysis (PCA)

Concept, algorithm, worked example, Python code and setup

1. What is PCA?

Principal Component Analysis (PCA) is an **unsupervised** machine learning technique for **dimensionality reduction**. It turns a dataset with many (often correlated) features into a smaller set of new features called **principal components**. The components are chosen so they keep as much of the original information, measured as **variance**, as possible.

Intuition: imagine photographing a 3D object. From a good angle, the 2D photo still shows most of its shape. PCA finds that "best angle" for your data: the directions along which the data spreads out most.

Each principal component is:

- a **linear combination** of the original features,
- **orthogonal** (perpendicular and uncorrelated) to every other component,
- ranked, so **PC1** captures the most variance, **PC2** the next most, and so on.

Common uses

- Reducing features before training a model (faster training, less overfitting).
- Visualizing high-dimensional data in 2D or 3D.
- Removing noise and redundancy caused by correlated features.
- Image compression and face recognition ("eigenfaces").

2. Key terms

Term	Meaning
Variance	How spread out a feature's values are. PCA treats variance as information.
Covariance	How two features change together. Positive means they rise together; negative means one rises as the other falls.
Covariance matrix	A $d \times d$ matrix holding the covariance of every pair of features (d = number of features).
Eigenvector	A direction that the covariance matrix only stretches, never rotates. In PCA each eigenvector is a principal component direction.
Eigenvalue (lambda)	How much variance lies along its eigenvector. Bigger means more important.
Explained variance ratio	$\lambda_i / \text{sum of all } \lambda$: the share of the total information that component i keeps.
Loadings	The weights of the original features inside each component.

3. The PCA algorithm, step by step

Input: a data matrix X with n samples (rows) and d features (columns), and the number of components k to keep ($k < d$).

Step 1: Standardize or center the data

Subtract each feature's mean so the data is centered at the origin. If features use different units (e.g. age in years and salary in rupees), also divide by the standard deviation. Otherwise the large-scale feature

dominates.

$$z = (x - \text{mean}) / \text{std}$$

Step 2: Compute the covariance matrix

$$C = (1 / (n - 1)) * X^T X \text{ (X is centered, C is } d \times d \text{)}$$

Step 3: Compute eigenvalues and eigenvectors of C

Solve the equation below. There are d pairs of eigenvalues and eigenvectors.

$$C v = \lambda v$$

Step 4: Sort the components

Sort the eigenvectors by their eigenvalues, largest first. The top eigenvector is PC1.

Step 5: Choose k

Keep the smallest k whose cumulative explained variance reaches your target, commonly 90-95%, or use the "elbow" in a scree plot (Section 5).

Step 6: Project the data

Stack the top k eigenvectors as columns of a $d \times k$ matrix W , then multiply:

$$X_{\text{reduced}} = X_{\text{centered}} W \text{ (} n \times d \text{)(} d \times k \text{) = (} n \times k \text{)}$$

In practice, libraries like scikit-learn compute PCA with **Singular Value Decomposition (SVD)** of the centered data instead of building the covariance matrix. It gives the same components and is more numerically stable.

4. Worked example (2D to 1D)

Ten points with two correlated features, X and Y. We reduce them to one dimension. These numbers are exactly what `main.py` prints in Part 1.

Point	1	2	3	4	5	6	7	8	9	10
X	2.5	0.5	2.2	1.9	3.1	2.3	2.0	1.0	1.5	1.1
Y	2.4	0.7	2.9	2.2	3.0	2.7	1.6	1.1	1.6	0.9

Step 1, center: mean X = 1.81, mean Y = 1.91. Subtract these from every point.

Step 2, covariance matrix:

$$C = \begin{bmatrix} 0.6166 & 0.6154 \\ 0.6154 & 0.7166 \end{bmatrix}$$

The large off-diagonal value (0.6154) shows that X and Y are strongly correlated, which is why one dimension can describe them.

Step 3-4, eigen-decomposition (sorted):

Component	Eigenvalue	Eigenvector	Explained variance
PC1	1.2840	[0.6779, 0.7352]	96.32%
PC2	0.0491	[-0.7352, 0.6779]	3.68%

Step 5, choose k = 1: PC1 alone keeps 96.3% of the information.

Step 6, project: each point becomes one number, e.g. point 1: $0.69 \times 0.6779 + 0.49 \times 0.7352 = 0.828$. All ten projected values:

$[0.828, -1.778, 0.992, 0.274, 1.676, 0.913, -0.099, -1.145, -0.438, -1.224]$

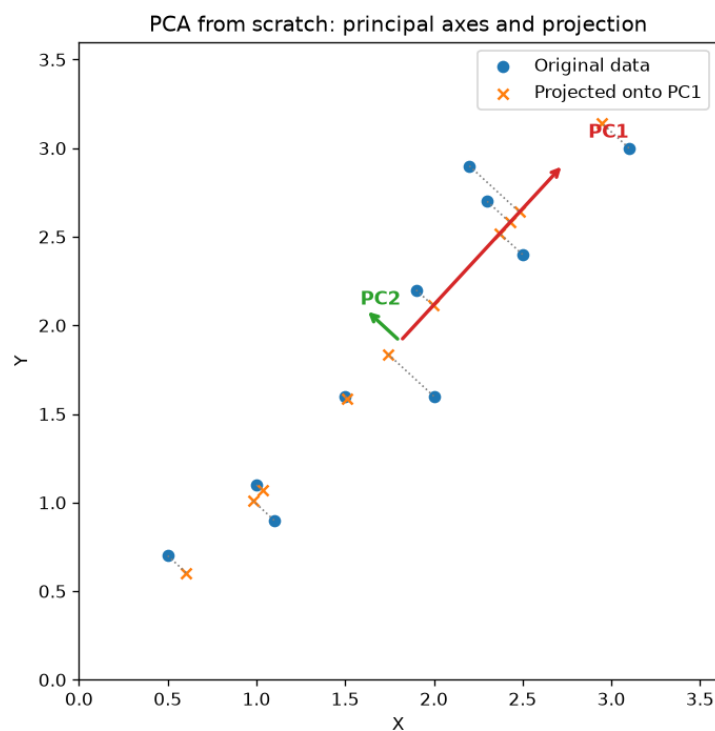


Figure 1. Red arrow = PC1 (direction of most spread), green = PC2. Orange crosses show where each point lands after projecting onto PC1; dotted lines are the information lost.

5. How many components to keep?

- **Cumulative explained variance:** keep enough components to reach about 90-95%.
- **Scree plot / elbow:** plot variance per component and stop where the curve flattens.
- **Purpose:** for visualization use $k = 2$ or 3 ; for model input, tune k with cross-validation.
- In scikit-learn, `PCA(n_components=0.95)` picks k automatically to keep 95%.

On the Iris dataset (4 features), PC1 explains 73.0% and PC2 22.9%, so **2 components keep 95.8%** of the information. The three flower species separate clearly in the 2D plot:

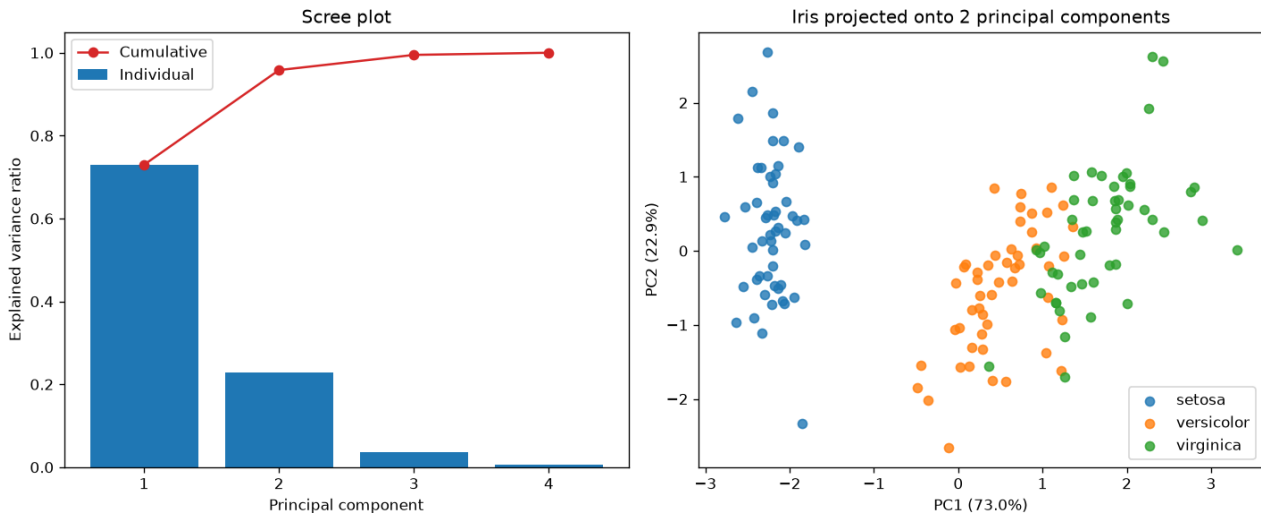


Figure 2. Left: scree plot with cumulative variance. Right: 150 Iris flowers projected from 4D to 2D.

Loadings for Iris show what each component means: PC1 is mostly petal length, petal width and sepal length (overall flower size), while PC2 is mostly sepal width.

	Sepal length	Sepal width	Petal length	Petal width
PC1	0.521	-0.269	0.580	0.565
PC2	0.377	0.923	0.024	0.067

6. Advantages, limitations and tips

Advantages	Limitations
Reduces features, so training is faster and uses less memory	Components are mixes of features, so they are harder to interpret
Removes correlation (multicollinearity) between features	Only captures linear relationships (use Kernel PCA, t-SNE or UMAP for non-linear data)
Can reduce noise and overfitting	Sensitive to feature scale, so you must standardize
Makes high-dimensional data easy to visualize	Sensitive to outliers, which inflate variance
No labels needed (unsupervised)	Maximizes variance, not class separation (LDA does that when labels exist)

Tips

- Always **standardize** first (`StandardScaler`) unless all features share a unit.

- Fit the scaler and PCA on **training data only**, then transform the test data, to avoid data leakage. A scikit-learn Pipeline does this for you.
- Eigenvector signs can flip between libraries ([0.68, 0.74] vs [-0.68, -0.74]). Both are correct.
- Use `pca.inverse_transform()` to map reduced data back to the original space.

7. Setup: requirements.txt and running the code

Project folder: MachineLearning/Unsupervised/PCA/

requirements.txt

```
numpy
pandas
scikit-learn
matplotlib
```

Package	Used for
numpy	Arrays, covariance matrix, eigen-decomposition (PCA from scratch)
pandas	Displaying data and loadings as tables
scikit-learn	PCA, StandardScaler and the Iris dataset
matplotlib	Scree plot and scatter plots

Steps to run (Windows, from the PCA folder)

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python main.py
```

The script prints every step to the terminal, saves `pca_scratch.png` and `pca_iris.png`, and opens the Iris charts in a window.

Quick version with scikit-learn only

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

X_scaled = StandardScaler().fit_transform(X)
pca = PCA(n_components=2) # or PCA(n_components=0.95)
X_reduced = pca.fit_transform(X_scaled)
print(pca.explained_variance_ratio_)
```

8. Full code: main.py

Part 1 implements PCA from scratch with NumPy (following Section 3) and checks the result against scikit-learn. Part 2 applies scikit-learn's PCA to the Iris dataset.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

np.set_printoptions(precision=4, suppress=True)

# =====
# PART 1: PCA from scratch (step by step) on a small 2D dataset
# =====

# Step 1: Create the dataset
data = {
    "X": [2.5, 0.5, 2.2, 1.9, 3.1, 2.3, 2.0, 1.0, 1.5, 1.1],
    "Y": [2.4, 0.7, 2.9, 2.2, 3.0, 2.7, 1.6, 1.1, 1.6, 0.9]
}
df = pd.DataFrame(data)
X = df.values
print("Step 1: Original data")
print(df, "\n")

# Step 2: Center the data (subtract the mean of each feature)
mean = X.mean(axis=0)
X_centered = X - mean
print("Step 2: Mean of each feature:", mean)
print("Centered data:\n", X_centered, "\n")

# Step 3: Covariance matrix
cov_matrix = np.cov(X_centered, rowvar=False)
print("Step 3: Covariance matrix:\n", cov_matrix, "\n")

# Step 4: Eigenvalues and eigenvectors of the covariance matrix
eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix)

# Step 5: Sort by eigenvalue (largest first)
order = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[order]
eigenvectors = eigenvectors[:, order]
print("Step 4-5: Eigenvalues (sorted):", eigenvalues)
print("Eigenvectors (columns = PC1, PC2):\n", eigenvectors, "\n")

# Step 6: Explained variance ratio
explained = eigenvalues / eigenvalues.sum()
print("Step 6: Explained variance ratio:", explained, "\n")

# Step 7: Project the data onto the top k components (k = 1)
k = 1
W = eigenvectors[:, :k]
X_projected = X_centered @ W
print(f"Step 7: Data projected onto top {k} component:\n", X_projected.ravel(), "\n")

# Step 8: Reconstruct back to 2D (shows the information that is kept)
X_reconstructed = X_projected @ W.T + mean

# Verify with scikit-learn (signs of components may be flipped - that is fine)
sk_pca = PCA(n_components=1).fit(X)
print("scikit-learn explained variance ratio:", sk_pca.explained_variance_ratio_)
print("scikit-learn PC1:", sk_pca.components_[0], "\n")

# Plot: original data, principal axes and projection
plt.figure(figsize=(6, 6))
plt.scatter(X[:, 0], X[:, 1], label="Original data", color="tab:blue")
plt.scatter(X_reconstructed[:, 0], X_reconstructed[:, 1],
            label="Projected onto PC1", color="tab:orange", marker="x")
for i in range(len(X)):
    plt.plot([X[i, 0], X_reconstructed[i, 0]], [X[i, 1], X_reconstructed[i, 1]],
```

```

        color="gray", linestyle=":", linewidth=1)
for j, color in zip(range(2), ["tab:red", "tab:green"]):
    vec = eigenvectors[:, j] * np.sqrt(eigenvalues[j]) * 1.2
    plt.annotate("", xy=mean + vec, xytext=mean,
                 arrowprops=dict(arrowstyle="->", color=color, linewidth=2))
    plt.text((mean + vec * 1.15), f"PC{j + 1}", color=color, fontsize=11, weight="bold")
plt.xlim(0, 3.6)
plt.ylim(0, 3.6)
plt.gca().set_aspect("equal")
plt.xlabel("X")
plt.ylabel("Y")
plt.title("PCA from scratch: principal axes and projection")
plt.legend()
plt.tight_layout()
plt.savefig("pca_scratch.png", dpi=120)

# =====
# PART 2: PCA with scikit-learn on the Iris dataset (4D -> 2D)
# =====

iris = load_iris()
X_iris, y_iris = iris.data, iris.target

# Step 1: Standardize (PCA is sensitive to feature scale)
X_scaled = StandardScaler().fit_transform(X_iris)

# Step 2: Fit PCA with all components to inspect explained variance
pca_full = PCA().fit(X_scaled)
print("Iris explained variance ratio:", pca_full.explained_variance_ratio_)
print("Cumulative:", np.cumsum(pca_full.explained_variance_ratio_), "\n")

# Step 3: Reduce to 2 components
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print("Iris shape before PCA:", X_iris.shape, "after PCA:", X_pca.shape)
print("Loadings (how much each feature contributes to each PC):")
print(pd.DataFrame(pca.components_, columns=iris.feature_names,
                  index=["PC1", "PC2"]).round(3), "\n")

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Scree plot
n = len(pca_full.explained_variance_ratio_)
axes[0].bar(range(1, n + 1), pca_full.explained_variance_ratio_, label="Individual")
axes[0].plot(range(1, n + 1), np.cumsum(pca_full.explained_variance_ratio_),
            marker="o", color="tab:red", label="Cumulative")
axes[0].set_xticks(range(1, n + 1))
axes[0].set_xlabel("Principal component")
axes[0].set_ylabel("Explained variance ratio")
axes[0].set_title("Scree plot")
axes[0].legend()

# 2D projection
for label, color in zip(range(3), ["tab:blue", "tab:orange", "tab:green"]):
    mask = y_iris == label
    axes[1].scatter(X_pca[mask, 0], X_pca[mask, 1], color=color,
                  label=iris.target_names[label], alpha=0.8)
axes[1].set_xlabel(f"PC1 ({{pca.explained_variance_ratio_[0]:.1%}})")
axes[1].set_ylabel(f"PC2 ({{pca.explained_variance_ratio_[1]:.1%}})")
axes[1].set_title("Iris projected onto 2 principal components")
axes[1].legend()

plt.tight_layout()
plt.savefig("pca_iris.png", dpi=120)
plt.show()

```

9. Summary

- PCA is an unsupervised dimensionality-reduction technique.
- It finds new, uncorrelated axes (principal components) ordered by how much variance they capture.
- Algorithm: standardize, covariance matrix, eigenvalues and eigenvectors, sort, choose k, project.
- Choose k with cumulative explained variance (about 95%) or a scree plot.

- Always scale features first, and fit PCA on training data only.